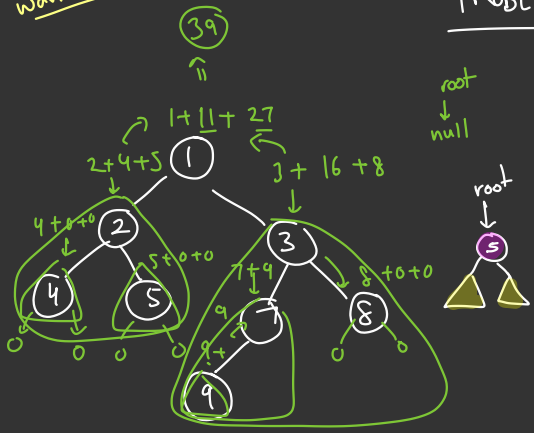
Warm Up

PROBLEMS ON TREES



1+2+3+4+5+7+8+9 = 39

```
int SumOfNodes (Node root) {
    if (root == null)
        return 0

    ans = root data +
          SumOfNodes (root left) +
          SumOfNodes (root right)

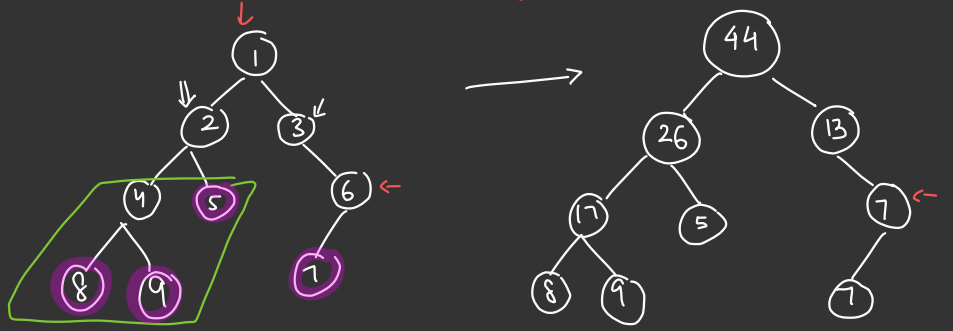
    return ans;
}
```

- Pre -
- In -
- Post -
- Level -

Warm-Up for you ?

Replace every node in the tree with sum of its descendants except leaf nodes which remain same

↳ all nodes in left & right subtree



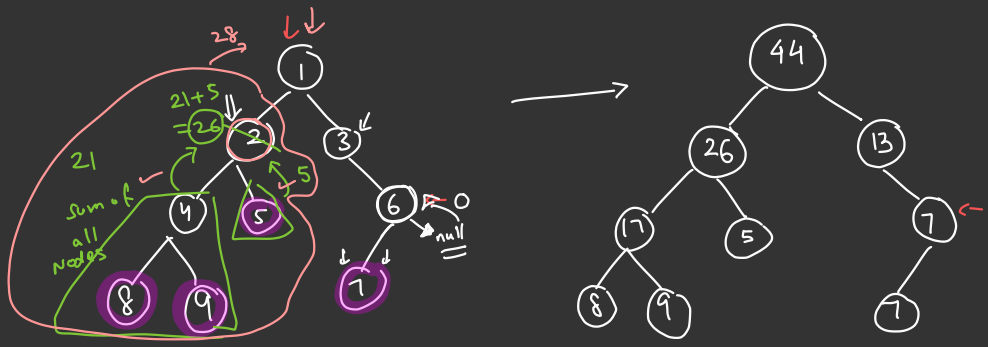
925 :)

```
int modifyTree (Node Root) {
    LS = sum(Root.left)      } O(N)
    RS = sum(Root.Right)
    Root.data = LS + RS
    modifyTree (Root.left)
    modifyTree (Root.Right)  ]
}
```

N Nodes

$O(N^2)$

Inefficient



2 ⟶ 26

⇒ root data = Left Sum + Right Sum

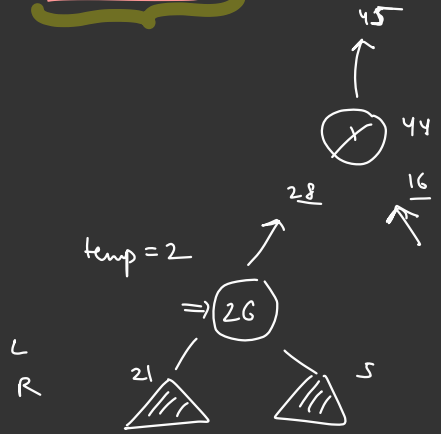
↓ Problem: you loose 2

```
int temp = root data,
root data = Left Sum + Right Sum
return root data + temp;
```

Code

```
int modifyTree (Node root)
    if (root == null)
        return 0
    if ( root is a leaf)
        return root data
// Rec case
LeftSum  = modifyTree (root.left)
RightSum = modifyTree (root.right)
```



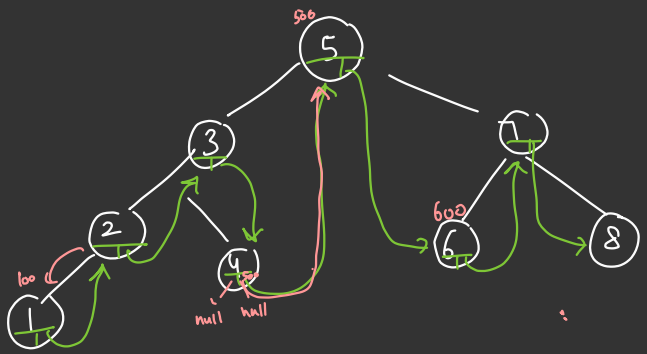


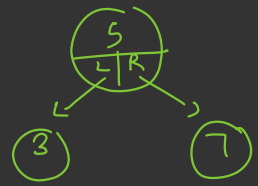
```
int temp = root data,
root data = leftSum + RightSum
return root data + temp;
}
```

Root

Convert BST Tree into a Sorted LinKed list (using same node objects)

- change 'left' as null for every node
- change 'right' of every node to next inorder successor





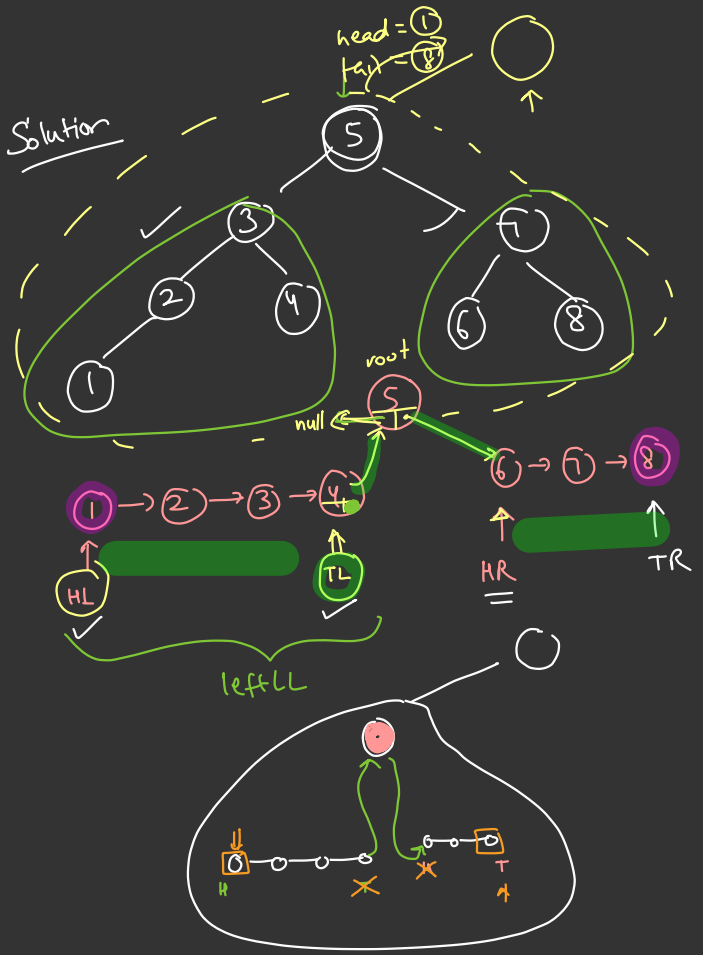


Linear structure (LinKed list)

```
class Pair{
    Node head;
    Node tail;

    Pair(h,t){
        head = h
        tail = t
    }
}
```

3



Pair Convert Tree 2 LL (Node root) {

Case - 4

// Both left & Right Subtree Exist

leftLL = Convert Tree 2 LL (root.left)

rightLL = Convert Tree 2 LL (root.right)

Post order

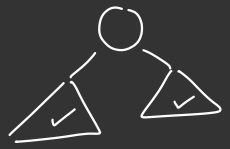
leftLL.tail.right = root

root.right = rightLL.head

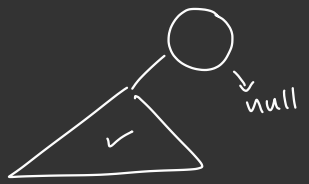
root.left = null

⇒ Return new Pair (leftLL.head, rightLL.tail)

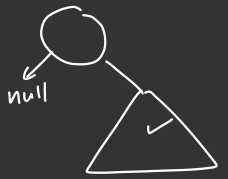
}



Case-4



Case-3



Case-2



Case-1

Case-1

```
if (root.left == null && root.right == null) {
    return new Pair(root, root);
}
```



Case-II

```
if (root.left == null && root.right != null) {
    Pair rightLL = ConvertTree2LL(root.right);
    root.right = rightLL.head;
```



return new Pair (root , rightLL tail);

}

Case-III

if (root.left != null && root.right == null) {

Pair leftLL = tree2LL (root.left)

leftLL.tail.right = root

root.left = null

return new Pair (leftLL.head , root)

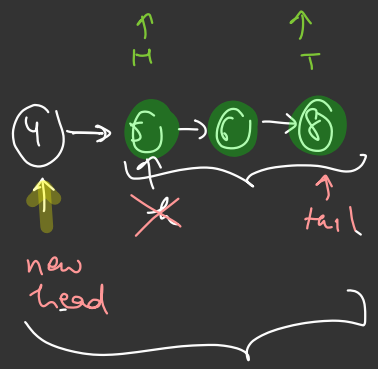
}

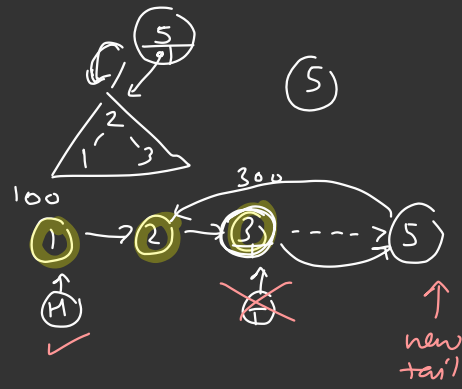
LL {

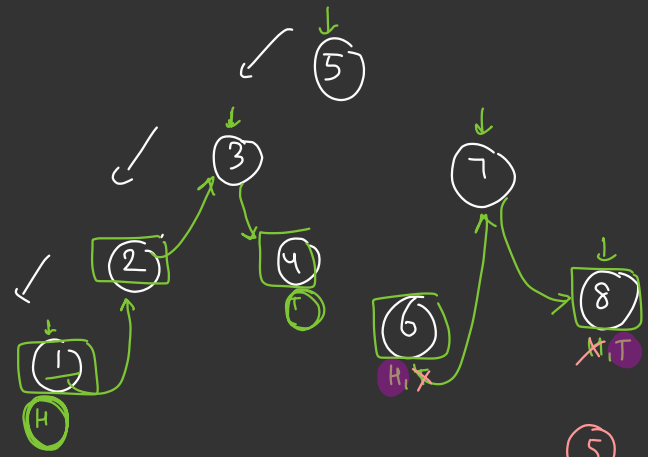
100 ⇒ 1

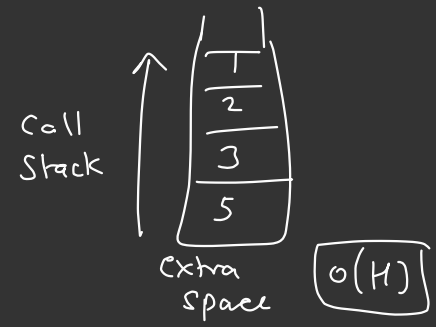
300 ⇒ 3 (tail) → Right

}

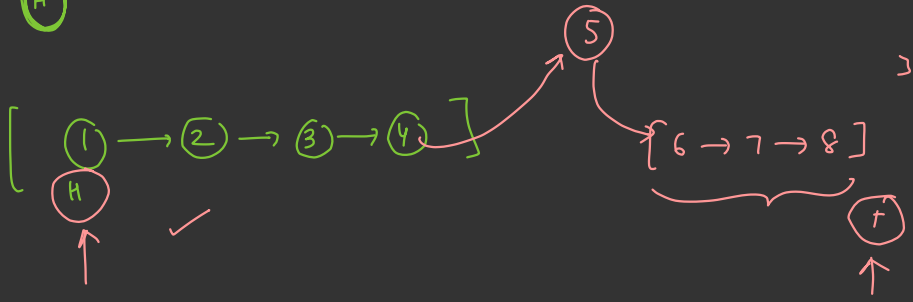




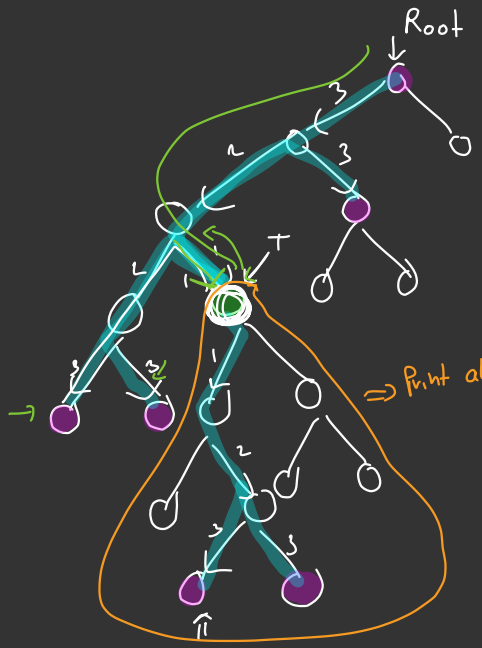




Main() {



}



⇒ Given Root, Target

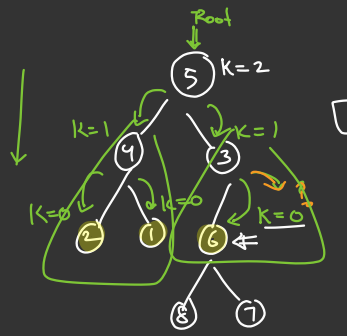
Print all nodes at disk 'k' edges from target

K=3

15 Mins



10 45



K=2

Part-1 Helper Fn

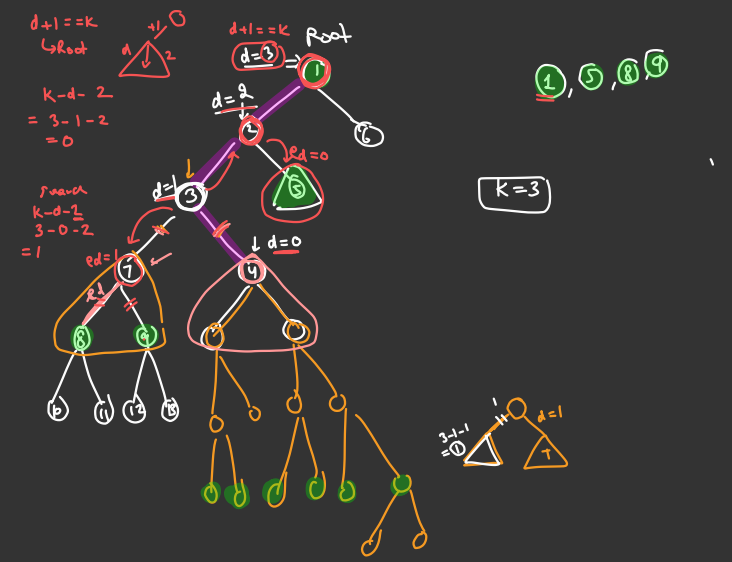
```
printAtLevelK ( root , K) {
    if (root == null)
        return
    if (K == 0)
        print (root data)
        return
    // Rec Case
    printAtLevelK (root.left, K-1)
    printAtLevel (root.right, K-1)
}
```

Root

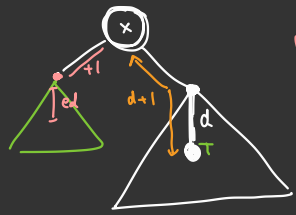
Pre-order

L

R







Rd + 1 + d + 1 = K

Rd = K - d - 2

```
int printNodesAtDistK(root,target,k){

        //base case
        if(root==null){
            return -1;
        }

        // target is found
        if(root==target){
            printKthLevelHelper(root,k);
            return 0;
        }

        // rec case
        dL = printNodesAtDistK(root.left,target,k);
        if(dl!=-1){
            if(dl + 1 == k){
                print(root.data);
            }
            else{
                printKthLevelHelper(root.right,k-d-2);
            }
            return 1 + dL;
        }

        dR = printNodesAtDistK(root.right,target,k);
        if(dR!=-1){
            if(dR + 1 == k){
                print(root.data)
            }
            else{
                printKthLevelHelper(root.left,k-d-2);
            }
            return 1 + dr;
        }

        return -1;
}
```

Sat - Morning (10:00 am) [optional]

Sat - evening (____) [Reg]